

Overview of Graph neural network

Subject: Graph neural network

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

1.

where $\mathbf{a}$ is a vector of learnable weights, $\cdot^T$ indicates transposition, $\mathbf{e}_{uv}$ are the edge features (if present), and LeakyReLU is a modified ReLU activation function. Attention coefficients are then normalized via softmax to make them easily comparable across different nodes. A GCN can be seen as a special case of a GAT where attention coefficients are not learnable, but fixed and equal to the edge weights w_{uv} .

2.

Graph-based representation of text helps to capture deeper semantic relationships between words. Many studies have used graph networks to enhance performance in various text processing tasks such as text classification, question answering, Neural Machine Translation (NMT), event extraction, fact verification, etc.

3.

To represent an image as a graph structure, the image is first divided into multiple patches, each of which is treated as a node in the graph. Edges are then formed by connecting each node to its nearest neighbors based on spatial or feature similarity. This graph-based representation enables the application of graph learning models to visual tasks. The relational structure helps to enhance feature extraction and improve performance on image understanding.

4.

where K is the number of attention heads, $\bigvee$ denotes vector concatenation, $\sigma(\cdot)$ is an activation function (e.g., ReLU), N_u is the set of immediate neighbor nodes of node u , including node u itself, α_{uv}^k are attention coefficients for the k -th attention head, and W^k is a matrix of trainable parameters for the k -th attention head. For the final GAT layer, the outputs from each attention head are averaged before the application of the activation function. Formally, the final GAT layer can be written as:

5.

$$\mathbf{h}_u = \bigvee_{k=1}^K \sigma \left(\sum_{v \in N_u} \alpha_{uv}^k W^k \mathbf{x}_v \right)$$

6.

$$\mathbf{h}_u = \phi(\mathbf{x}_u, \bigoplus_{v \in N(u)} \psi(\mathbf{x}_u, \mathbf{x}_v, \mathbf{e}_{uv})) \quad \{\displaystyle \mathbf{h}_u = \phi(\mathbf{x}_u, \bigoplus_{v \in N(u)} \psi(\mathbf{x}_u, \mathbf{x}_v, \mathbf{e}_{uv}))\}$$

7.

Graph attention network The graph attention network (GAT) was introduced by Petar Veličković et al. in 2018. A graph attention network is a combination of a GNN and an attention layer. The implementation of attention layer in graphical neural networks helps provide attention or focus to the important information from the data instead of focusing on the whole data. A multi-head GAT layer can be expressed as follows:

8.

where $\mathbf{i} = \text{top}_k(\mathbf{y})$ is the subset of nodes with the top-k highest projection scores, $\odot$ denotes element-wise matrix multiplication. The self-attention pooling layer can be seen as an extension of the top-k pooling layer. Differently from top-k pooling, the self-attention scores computed in self-attention pooling account both for the graph features and the graph topology.

9.

where ϕ and ψ are differentiable functions (e.g., artificial neural networks), and $\bigoplus$ is a permutation invariant aggregation operator that can accept an arbitrary number of inputs (e.g., element-wise sum, mean, or max). In particular, ϕ and ψ are referred to as update and message functions, respectively. Intuitively, in an MPNN computational block, graph nodes update their representations by aggregating the messages received from their neighbours. The outputs of one or more MPNN layers are node representations $\mathbf{h}_u$ for each node $u \in V$ in the graph. Node representations can be employed for any downstream task, such as node/graph classification or edge prediction. Graph nodes in an MPNN update their representation aggregating information from their immediate neighbours. As such, stacking n MPNN layers means that one node will be able to communicate with nodes that are at most n "hops" away. In principle, to ensure that every node receives information from every other node, one would need to stack a number of MPNN layers equal to the graph diameter. However, stacking many MPNN layers may cause issues such as oversmoothing and oversquashing. Oversmoothing refers to the issue of node representations becoming indistinguishable. Oversquashing refers to the bottleneck that is created by squeezing long-range dependencies into fixed-size representations. Countermeasures such as skip connections (as in residual neural networks), gated update rules and jumping knowledge can mitigate oversmoothing. Modifying the final layer to be a fully-adjacent layer, i.e., by considering the graph as a complete graph, can mitigate oversquashing in problems where long-range dependencies are required. Other "flavours" of MPNN have been developed in the literature, such as graph convolutional networks and graph attention networks, whose definitions can be expressed in terms of the MPNN formalism.

10.

where $\mathbf{H}$ is the matrix of node representations $\mathbf{h}_u$, $\mathbf{X}$ is the matrix of node features $\mathbf{x}_u$, $\sigma(\cdot)$ is an activation function (e.g., ReLU), $\tilde{\mathbf{A}}$ is the graph adjacency matrix with the addition of self-loops, $\tilde{\mathbf{D}}$ is the graph degree matrix with the addition of self-loops, and Θ is a matrix of trainable parameters. In particular, let $\mathbf{A}$ be the graph adjacency matrix: then, one can define $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ and $\tilde{\mathbf{D}}_{ii} = \sum_{j \in V} \tilde{\mathbf{A}}_{ij}$, where $\mathbf{I}$ denotes the identity matrix. This normalization ensures that the eigenvalues of $\tilde{\mathbf{D}} - \frac{1}{2} \tilde{\mathbf{A}} \tilde{\mathbf{D}} - \frac{1}{2} \tilde{\mathbf{A}}$ are bounded in the range $[0, 1]$, avoiding numerical instabilities and exploding/vanishing gradients. A limitation of GCNs is that they do not allow multidimensional edge features $\mathbf{e}_{uv}$. It is however possible to associate scalar weights w_{uv} to each edge by imposing $A_{uv} = w_{uv}$, i.e., by setting each nonzero entry in the adjacency matrix equal to the weight of the corresponding edge.